Additional Experiments

The effect of $\Delta^\star$ on the regret:

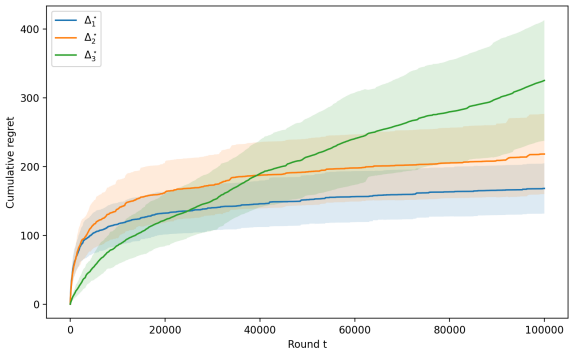
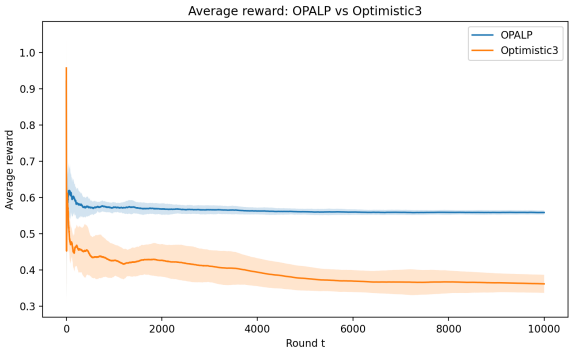
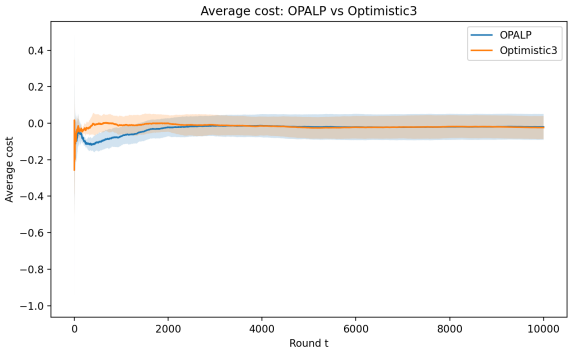


Figure 2. We assess the effect of $\Delta^\star$ on the regret based on **Example 1** averaged over 30 runs. We vary the values of the gap such that $\Delta_1^\star > \Delta_2^\star \gg \Delta_3^\star$. We observe that the regret scales inversely with $\Delta^\star$. In particular, when $\Delta^\star \to 0$ (very small values), the regret behavior changes, transitioning from a problem-dependent (logarithmic) regime to a worst-case regime.

OPALP vs Optimistic^3:



(a)



(b)

Figure 3. Comparison between **OPALP** and **Optimistic**^{3} (Guo et al., 2026) on the same experimental setup used in their work. We observe that both algorithms achieve comparable average costs, while **OPALP** attains a higher average reward than **Optimistic**^{3}, consistent with our theoretical claims. The observed fluctuations are explained by the fact that the algorithms occasionally produce infeasible policies.

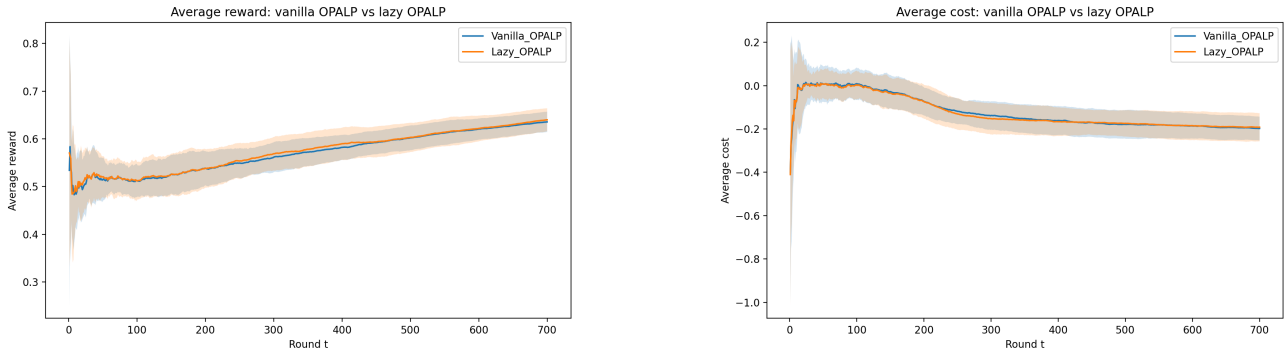
Computational complexity:

Computational burden:

Algorithm / $\lvert\mathcal{C}\rvert$	10	100	500	1000	5000	10000
OPALP	$5 \cdot 10^{-4}$	$13 \cdot 10^{-4}$	$11 \cdot 10^{-3}$	$25 \cdot 10^{-3}$	0.5	4
Optimistic^3	$5 \cdot 10^{-4}$	$7 \cdot 10^{-4}$	$3 \cdot 10^{-3}$	$8 \cdot 10^{-3}$	0.08	0.2

Table 2. We empirically compare the computational complexity of **OPALP** with the recent method **Optimistic**^{3} of Guo et al. (2024). We fix the number of arms to $|\mathcal{A}| = 10$ and vary the number of contexts $|\mathcal{C}| \in \{10, 100, 500, 1000, 10000\}$. We report the average per-round runtime (in seconds). We observe that the computational burden of **OPALP** increases significantly as the decision space grows, compared to **Optimistic**^{3}.

Vanilla OPALP vs Lazy OPALP (Figure 4):



(a) *(b)*

Figure 4. To mitigate the computational burden of **OPALP** in large decision spaces, we propose a **lazy OPALP** variant that updates the policy only occasionally, resulting in only $\mathcal{O}(\log T)$ LP solves. We compare this approach with the vanilla algorithm on an instance with 20 contexts and 10 arms, using random Gaussian distributions and averaging over 20 runs. We observe no significant performance deterioration, consistent with the theoretical insights.

OPALP and Contextual Bandits with Knapsack (CBwK)

Algorithm / Budget	300	500	700	900
OPALP	298.2	484.7	660.3	838.9
Agrawal and Devanur (2016)	307.8	489.6	666.2	840.0

Table 3. We instantiate **OPALP** for the CBwK setting as described in the paper (using a stopping time and incorporating the budget into the cost samples), with 4 arms and 3 contexts. We consider different budget values with $T = 2000$ and average over 20 runs. We observe that **OPALP** achieves performance comparable to the algorithm specifically designed for the CBwK setting.